

Informe de Arquitectura y Orquestación: Chatbot GeoSalud

Este documento detalla la estructura técnica, flujo de datos y orquestación del chatbot inteligente integrado en el **Observatorio GeoSalud del Valle del Cauca**. El chatbot ofrece una experiencia híbrida interactiva combinando respuestas rápidas basadas en reglas locales (sin consumo de API externa) y consultas complejas basadas en Modelos de Lenguaje Grande (LLMs) con la suite ADK de Google.

1. Diagrama de Flujo y Orquestación General

El siguiente diagrama en formato Mermaid representa cómo interactúan el cliente (frontend), el servidor intermedio (FastAPI) y la capa de agentes inteligentes (ADK con Gemini):

```
<div style="page-break-inside: avoid; margin-top: 15px; margin-bottom: 25px; border: 1px solid #cbd5e1; padding: 10px; border-radius: 5px;">
  <div style="text-align: center; font-size: 11pt; font-weight: bold; margin-bottom: 12px; color: #1e1b4b; text-typography: none;">
    Diagrama de Flujo y Orquestación del Asistente
  </div>
  <table style="width: 100%; border-spacing: 10px; border-collapse: separate;">
    <!-- Capa Cliente -->
    <tr>
      <td colspan="2" style="background-color: #0f172a; color: #f8fafc; border: 1.5px solid #38bdf8; padding: 10px; vertical-align: top;">
        <div style="font-size: 9.5pt; font-weight: bold; margin-bottom: 3px;">CAPA CLIENTE (NAVEGADOR)</div>
        <div style="font-size: 8pt; color: #94a3b8; line-height: 1.3;">
          Interfaz Web &bull; Widget de Chat (geodata-chatbot.js) &bull; Aplicador (geodata-app.js)
        </div>
      </td>
    </tr>
    <tr>
      <td colspan="2" style="background-color: #f1f3f4; padding: 10px; vertical-align: top;">
        <div style="font-size: 8pt; color: #64748b; padding: 5px 0 0 20px;">
          &darr; 1. Envía consulta del usuario &amp; session_id &darr;
        </div>
      </td>
    </tr>
    <!-- Servidor Backend -->
    <tr>
      <td colspan="2" style="background-color: #1e1b4b; color: #f8fafc; border: 1.5px solid #818cf8; padding: 10px; vertical-align: top;">
        <div style="font-size: 9.5pt; font-weight: bold; margin-bottom: 3px;">SERVIDOR BACKEND (FastAPI - s</div>
        <div style="font-size: 8pt; color: #cbd5e1; line-height: 1.3;">
          API REST (POST /chat) &bull; Enrutador Híbrido &bull; Gestor de Caché
        </div>
      </td>
    </tr>
  </table>
</div>
```

```

        </div>
    </td>
</tr>
<!-- Bifurcación Decision -->
<tr>
    <td style="width: 50%; background-color: #f8fafc; border: 1.5px solid #cbd5e1; border-left: 4px solid #
        <div style="font-size: 8.5pt; font-weight: bold; color: #1e293b; margin-bottom: 4px;">A. INTERCEPTA
        <div style="font-size: 7.5pt; color: #475569; line-height: 1.3;">
            Resuelve al instante con bases de datos locales (PostgreSQL, DANE, CSV):
            <ul style="margin-top: 3px; margin-bottom: 0; padding-left: 12px;">
                <li>Población y ciclos de vida</li>
                <li>Tops / Rankings por año</li>
                <li>Series de tiempo sin año</li>
                <li>Acciones de navegación directa</li>
            </ul>
        </div>
    </td>
    <td style="width: 50%; background-color: #022c22; border: 1.5px solid #064e3b; border-left: 4px solid #
        <div style="font-size: 8.5pt; font-weight: bold; color: #34d399; margin-bottom: 4px;">B. CAPA DE AG
        <div style="font-size: 7.5pt; color: #a7f3d0; line-height: 1.3; margin-bottom: 6px;">
            Consultas complejas o interpretación de datos:
        </div>

    <table style="width: 100%; border-spacing: 3px; border-collapse: separate;">
        <tr>
            <td style="background-color: #064e3b; border: 0.5px solid #10b981; padding: 3px; border-rad
                <strong>epidemia</strong><br>SQL DB
            </td>
            <td style="background-color: #064e3b; border: 0.5px solid #10b981; padding: 3px; border-rad
                <strong>viz</strong><br>Plots
            </td>
            <td style="background-color: #064e3b; border: 0.5px solid #10b981; padding: 3px; border-rad
                <strong>ui</strong><br>Actions
            </td>
        </tr>
    </table>

    <div style="font-size: 7pt; color: #cbd5e1; margin-top: 6px; text-align: center;">
        Llaman a: <code>tools.py</code>, <code>viz_tools.py</code>, <code>ui_tools.py</code>
    </div>
</td>
</tr>

```

```

        <tr>
            <td colspan="2" style="text-align: center; font-size: 9pt; color: #64748b; padding: 0; margin: 0; line-
                &uarr; Retorna: Respuesta JSON + Acciones de Interfaz + Gráficas (Artifacts) &uarr;
            </td>
        </tr>
    </tr>

    <!-- Retorno al Cliente -->
    <tr>
        <td colspan="2" style="background-color: #0f172a; color: #f8fafc; border: 1.5px solid #38bdf8; padding:
            <div style="font-size: 9.5pt; font-weight: bold; margin-bottom: 3px;">APLICACIÓN EN PANTALLA</div>
            <div style="font-size: 8pt; color: #94a3b8; line-height: 1.3;">
                Se renderiza el texto y la imagen de tendencias &bull; <code>geodata-app.js</code> actualiza el
            </div>
        </td>
    </tr>
</table>
</div>

```

2. Componentes de la Arquitectura

La arquitectura se divide en tres capas principales:

A. Capa de Presentación (Frontend)

- **Geodata Salud.html:** Contiene el diseño y estructura visual. El chatbot se aloja en un contenedor flotante (`#s-chatbot.section`) que se activa a través de un botón flotante (FAB) o al navegar a la pestaña "Asistente IA".
- **geodata-chatbot.js:** Controla el ciclo de vida del chat en el navegador. Maneja la persistencia de la sesión mediante `sessionStorage`, realiza el pooling de conexión, envía peticiones HTTP POST a `/chat`, formatea mensajes markdown en burbujas de diálogo y renderiza las imágenes (gráficas) generadas como *artifacts*.
- **geodata-app.js:** Implementa la función `applyGeoSaludChatAction(action)`. El chatbot puede devolver una lista de "acciones de UI" (ej. cambiar de pestaña, prender o apagar Cali, modificar la métrica en el mapa) y esta función las aplica directamente al estado global de la aplicación (actualizando mapas Leaflet e invalidando dimensiones).

B. Capa de Servidor y Enrutamiento Híbrido (FastAPI Backend)

Ubicada en `server.py`, esta capa expone la API REST que consume el frontend y aplica una estrategia de enrutamiento y procesamiento híbrido inteligente:

1. **Extracción de Acciones Secuenciales (`_extract_navigation_action`):** Ante una consulta mixta (ej. "Abre demografía y dime la población de Cali"), el servidor extrae y aísla la intención de navegación (por ejemplo, `{"type": "navigate", "section": "demografia"}`), eliminando los verbos de navegación y conectores coordinantes (y, e) del mensaje. De esta forma, el backend aísla el mensaje restante ("dime la población de Cali") para su procesamiento analítico.
2. **Preprocesamiento y Coincidencia Difusa (`_preprocess_message`):** Aplica coincidencia difusa (`difflib.get_close_matches`) y mapeo de sinónimos/alias (ej. "buga" -> "Guadalajara de Buga", "San Santiago de Cali" -> "Cali") para corregir errores tipográficos en los municipios ingresados por el usuario. También sanitiza de forma automática caracteres corruptos provenientes de la codificación de la base de datos (por ejemplo, traduciendo ALCAL? a ALCALA o TULU? a TULUA), previniendo fallos en las consultas SQL de las herramientas.

Interceptación Local (`_local_response`): Antes de consumir la API del LLM (ahorrando tiempo y cuota), el servidor analiza el mensaje limpio usando reglas sintácticas y expresiones regulares optimizadas:

- `_local_poblacion_municipio_response`: Resuelve consultas de población general, por sexo y ciclo de vida de manera instantánea.
 - `_local_top_municipios_response`: Si el usuario pide un ranking, dibuja la gráfica con Matplotlib y calcula el texto de respuesta directamente desde la base de datos cargada.
 - `_local_series_top_municipios_response` / `_local_followup_response`: Genera series de tiempo y permite excluir municipios de manera fluida ("sin Cali", "quitar Palmira").
 - `_local_serie_municipio_response` (Consultas de Tendencia sin Año): Si el usuario consulta tendencias temporales para un municipio específico sin indicar año (ej. "muestrame los casos de dengue de cali"), el sistema intercepta esta consulta, genera la gráfica de la serie histórica y despacha la navegación correspondiente a tendencias.
 - `_local_observatory_action_response`: Traduce intenciones de navegación directa en eventos de interfaz.
4. **Capa del Agente ADK (`runner.run_async`):** Si la consulta no es mapeable localmente (ej: requiere interpretación abierta o análisis multivariable de dengue y clima), el servidor delega al agente de inteligencia artificial de ADK con el mensaje limpio.
 5. **Encadenamiento de Acciones (Action Chaining):** Al armar el ChatResponse final (tanto para respuestas locales como basadas en el agente), si se extrajo una acción de navegación en el paso 1, esta se antepone al inicio de la lista de acciones (`actions.insert(0, nav_action)`), logrando una transición de UI secuencial.
 6. **Manejo de Cuotas / Fallback:** Si el agente de ADK arroja un error de cuota o rate limit (429 / RESOURCE_EXHAUSTED), el backend intenta resolver la pregunta usando el procesador local, asegurando disponibilidad permanente del chatbot de forma offline.

IMPORTANT: **Ruteo Secuencial (Action Chaining):** La función `_extract_navigation_action` en `server.py` remueve de forma limpia los conectores del lenguaje natural para asegurar que el motor de

inferencia (o local) no sufra ruido con palabras sueltas. En el cliente, la acción de navegación se ejecuta de manera prioritaria para que el cambio de sección se visualice antes de procesar otras actualizaciones de componentes.

C. Capa de Inteligencia Artificial (Google ADK & Gemini)

Construida sobre el **Agent Development Kit (ADK) de Google**, la lógica de agentes se orquesta en la carpeta `chatbot/observatorio_agent/`: * **Orquestador Central (`agent.py`)**: * Define el agente raíz `observatorio_orchestrator`. * Utiliza `gemini-2.5-flash-lite` como motor cognitivo. * Su tarea es clasificar la intención del usuario y transferir la conversación a uno de los tres agentes especialistas mediante el patrón *Sub-Agentes*.

3. Detalle y Funcionamiento de los Sub-Agentes

Cada uno de los tres sub-agentes cuenta con instrucciones detalladas, memoria de estado (`state` de sesión) y un conjunto de herramientas especializadas para resolver la tarea en su área de competencia:

A. Agente de Epidemiología (`epidemio_agent`)

- **Archivo:** `epidemio_agent.py`
- **Propósito:** Especialista numérico de dengue y demografía. Responde preguntas sobre conteos de casos, tasas de incidencia, pirámides poblacionales, etapas del ciclo de vida y distribución por género.

Funcionamiento:

- Recibe la consulta del orquestador y decide qué herramienta utilizar para obtener datos precisos de la base de datos PostgreSQL.
- Gestiona la memoria de sesión (`state.last_municipio`, `state.last_anio`, `state.metrica_preferida`) para resolver consultas implícitas (ej. "¿Y en Palmira?" → usa el último año y métrica preferidos).
- Formatea los resultados numéricos en prosa clara y tablas o listas formateadas en Markdown, citando siempre la procedencia del dato (municipio + año).

B. Agente de Visualización (`viz_agent`)

- **Archivo:** `viz_agent.py`
- **Propósito:** Especialista en análisis visual y graficación. Genera gráficos estadísticos a partir de los datos históricos.

Funcionamiento:

- Se activa ante palabras clave de intención gráfica (ej. "grafica", "scatter", "histograma", "ver evolución").

- Llama a la herramienta gráfica adecuada, la cual genera la imagen en memoria y la almacena en el sistema de **Artifacts** de ADK como un archivo PNG.
- Implementa un **caché de sesión** para reutilizar gráficos idénticos que ya hayan sido generados, evitando reprocesamientos costosos.
- Escribe 1 o 2 líneas en chat explicando el hallazgo principal ilustrado en la gráfica (por ejemplo, el año de mayor pico o el municipio que encabeza el gráfico).

C. Agente de Interfaz (**ui_agent**)

- **Archivo:** **ui_agent.py**
- **Propósito:** Especialista en el control reactivo del panel web. Sincroniza la vista física del usuario con la conversación.

Funcionamiento:

- Traduce intenciones de manipulación del sitio web (ej. *"abre priorización"*, *"pon el mapa del Valle en tasa de incidencia"*, *"quita a Cali del mapa"*) en payloads JSON estructurados.
- Confirma la acción al usuario de forma muy breve e inyecta la acción a la respuesta HTTP, para que el frontend (geodata-app.js) actualice el mapa Leaflet, recalcule KPIs o active pestañas inmediatamente.

4. Detalle de las Herramientas Desarrolladas (Tools)

Las herramientas (*FunctionTools*) están organizadas en módulos de Python según su tipo. Todas ellas siguen el protocolo ADK (aceptan `ToolContext`, reciben parámetros de tipo básico, manejan excepciones y retornan diccionarios con un estado `"success"` o `"error"`).

A. Herramientas de Base de Datos y Demografía (**tools.py**)

Módulo principal de consultas analíticas y demográficas que conecta a la base de datos PostgreSQL/PostGIS: *

- `listar_municipios`: Obtiene los nombres oficiales de los 42 municipios del Valle del Cauca para corregir o validar entradas de usuario. *
- `casos_por_municipio_anio`: Extrae los casos confirmados, la población y la tasa de incidencia de dengue para un municipio y un año dados. *
- `top_municipios`: Calcula el ranking de los N municipios más afectados en un año según casos absolutos o tasa de incidencia. *
- `serie_temporal_municipio`: Obtiene la evolución histórica anual (2019-2026) de un municipio en particular. *
- `resumen_anio`: Genera estadísticas acumulativas a nivel de todo el departamento del Valle del Cauca para un año. *
- `consultar_poblacion_municipio`: Obtiene la población total de un municipio (año por defecto: 2024), detallando cantidad de hombres, mujeres y su porcentaje correspondiente. *
- `consultar_poblacion_ciclo_vida`: Devuelve la población segmentada por las 6 etapas del ciclo de vida (Primera infancia, Infancia, Adolescencia, Juventud, Adultez y Persona Mayor). *
- `consultar_piramide_poblacional`: Extrae la población segmentada por rangos de edad quinquenales y sexo, ideal para construir la pirámide poblacional del municipio. *
- `establecer_preferencia` y

`mostrar_contexto`: Utilidades para persistir y recuperar preferencias del usuario en el estado de sesión (por ejemplo, trabajar siempre con tasa de incidencia).

B. Herramientas de Visualización y Gráficos (`viz_tools.py`)

Módulo encargado de graficar los datos con `seaborn` y `matplotlib` y guardarlos como PNG en el almacenamiento de artifacts: * `grafica_casos_por_anio`: Dibuja la tendencia del departamento completo a lo largo del periodo 2019-2026. * `grafica_top_municipios`: Gráfico de barras horizontales con el ranking de un año determinado. * `grafica_top_municipios_todos_anios`: Gráfico que muestra la evolución acumulativa del ranking a lo largo del tiempo. * `grafica_serie_municipio`: Genera un gráfico de línea temporal para un único municipio. * `grafica_series_municipios`: Dibuja múltiples líneas temporales simultáneas para comparar de 2 a 4 municipios específicos. * `grafica_facet_municipios`: Crea una grilla comparativa (subplots o paneles independientes) cuando se analizan más de 4 municipios a la vez. * `grafica_poblacion_vs_incidencia`: Genera un diagrama de dispersión (scatter plot) para observar la correlación entre el tamaño poblacional y la tasa de incidencia de dengue. * *Mecanismo de caché incorporado*: Todas las herramientas consultan la función interna `_get_or_create_artifact` para verificar si un archivo con idénticos parámetros ya existe en sesión y así retornar su ID de forma instantánea.

C. Herramientas de Control de Interfaz (`ui_tools.py` y `ui_actions.py`)

Encargadas de generar y despachar las acciones hacia la pantalla: * `abrir_modulo_observatorio`: Cambia el módulo activo de la aplicación en el navegador (Dashboard, Indicadores, Tendencias, Priorización, Geovisor, Demografía). * `mostrar_mapa_coropletico`: Actualiza los filtros globales del mapa coroplético (año, variable: casos/incidencia/riesgo/delta, e inclusión/exclusión de Cali). * `mostrar_geovisor`: Configura el geovisor interactivo de Leaflet en sus modos visuales (burbujas, calor o coroplético) con sus respectivas variables. * *Generadores de Acciones (`ui_actions.py`)*: Contiene las funciones estructuradas `navigate_action`, `coropletico_action` y `geovisor_action` que mapean directamente los JSON de comando que ejecuta `applyGeoSaludChatAction` en el frontend.

5. Entorno de Ejecución en Vivo y Resolución de Conflictos

Activación del Servidor

Para activar el servidor FastAPI en el entorno de desarrollo local, se ejecuta la suite de Python con `uvicorn`:

```
# Navegar al directorio del chatbot
cd observatorio_geosalud/chatbot

# Activar el entorno virtual local y arrancar el servidor
..\venv\Scripts\python server.py
```

NOTE: El puerto por defecto es el ``8080`` pero se puede configurar libremente mediante la variable de entorno ``CHATBOT_PORT`` del archivo ``.env`` del observatorio.

Diagnóstico de Puertos y Procesos Huérfanos en Windows

En sistemas Windows, el proceso supervisor de recarga de uvicorn (reload=True) puede en ocasiones quedar en un estado inconsistente al cambiar de terminales o editar código sensible del backend, dejando subprocesos "zombies" que siguen reteniendo el puerto de escucha 8080 de manera invisible en memoria.

Si se produce un error del tipo `OSError: [Errno 10048] Only one usage of each socket address is normally permitted`:

1. **Identificar el PID del proceso reteniendo el puerto:** `powershell netstat -ano | findstr :8080`
2. **Forzar la finalización de los procesos mediante PowerShell:** `powershell Stop-Process -Id <PID> -Force`
3. **Reiniciar el servidor:** Ejecutar nuevamente el comando de arranque para iniciar el servicio limpio.

6. Protocolo de Comunicación (Formato de Datos)

Petición del Cliente

El widget de chat envía un objeto JSON con el mensaje y el identificador de sesión:

```
{
  "session_id": "session_1779458921000_3ab4e",
  "message": "Abre el módulo de demografía y muéstrame la población de Cali"
}
```

Respuesta del Servidor (Caso Mixto / Action Chaining)

El servidor procesa los requerimientos mixtos e inserta de forma secuencial las acciones. En el caso de *"Navega a tendencias y grafica la tendencia de dengue de Cali"*, la respuesta resultante combina el cambio de pestaña y el renderizado gráfico:

```
{
  "reply": "Listo. Mostrando la serie de tiempo para Cali en el módulo de Tendencias.\n\n**Serie histórica de dengue",
  "artifacts": [
    "serie_historica_casos_cali.png"
  ],
  "session_id": "session_1779458921000_3ab4e",
  "actions": [
    {
      "type": "navigate",
      "section": "tendencias",
      "anio": null
    },
    {
      "type": "show_tendencias",
      "municipios": ["CALI"],

```



```
    "metrica": "casos",  
    "anio": null  
  }  
]  
}
```

7. Ventajas del Diseño Actual

1. **Soporte de Intenciones Mixtas (Action Chaining):** Permite que el usuario realice comandos complejos en un solo mensaje (ej. navegar a una sección y además solicitar datos/gráficos). El backend los desenreda de manera determinista y garantiza que la navegación ocurra en primer lugar antes de que el frontend intente actualizar los widgets específicos de esa sección.
2. **Cero Latencia en Comandos Comunes:** Al usar el enrutador local para consultas demográficas directas, navegación y rankings básicos, el usuario recibe respuestas visuales y numéricas en milisegundos sin consumir cuota del LLM.
3. **Coincidencia Difusa Robusta (Fuzzy Matching):** Protege la robustez del chatbot frente a fallos tipográficos ("buga", "cali con calima") y desajustes de codificación de base de datos ("ALCAL?"), normalizándolos a valores canónicos válidos antes de pasarlos a las consultas SQL y visualizaciones.
4. **Optimización de Tendencias sin Año:** Resuelve de manera local e interactiva consultas donde el usuario pide tendencias o series históricas de un municipio de forma abierta, sin fallar o solicitar años adicionales innecesariamente.
5. **Persistencia del Contexto (State):** Los sub-agentes recuerdan variables clave de sesión (como el último municipio mencionado, el año de análisis o la métrica preferida), lo que posibilita una conversación con contexto continuo (ej. "¿Y en Palmira?").
6. **Resiliencia ante Caídas del Modelo:** Si el agente externo de ADK/Gemini sufre de cuotas excedidas o fallos de red, el pipeline local intercepta y responde la consulta de manera autónoma con los datos cargados localmente en memoria.